

AstraNotes

UML Design Package

Jonathan Fong · CSEN 296B · Prof. Paddu Melanahalli

Spring 2025

1. Class Diagram

The structural backbone of AstraNotes. Refined from the Week 4.1 foundation to reflect the actual C++17 design: **NoteManager** owns notes as `map<UUID, unique_ptr<Note>>` for $O(1)$ lookup (NFR-1), holding **NoteFactory** and **StorageInterface** as injected references rather than owned pointers — the injection boundary makes both replaceable with mocks (SPR-2). **VersionHistory** is composed inside each **Note** instance, not in **NoteManager**, so history lifetime is tied to the note that owns it. **EncryptionEngine** is a pure virtual interface; **AESEngine** is the concrete AES-256-GCM realization, keeping crypto logic out of **SecureNote**'s public API (SPR-2). **CLView** is a thin «boundary» class that depends on **NoteManager**, completing the MVC triple without leaking domain logic into the view layer.

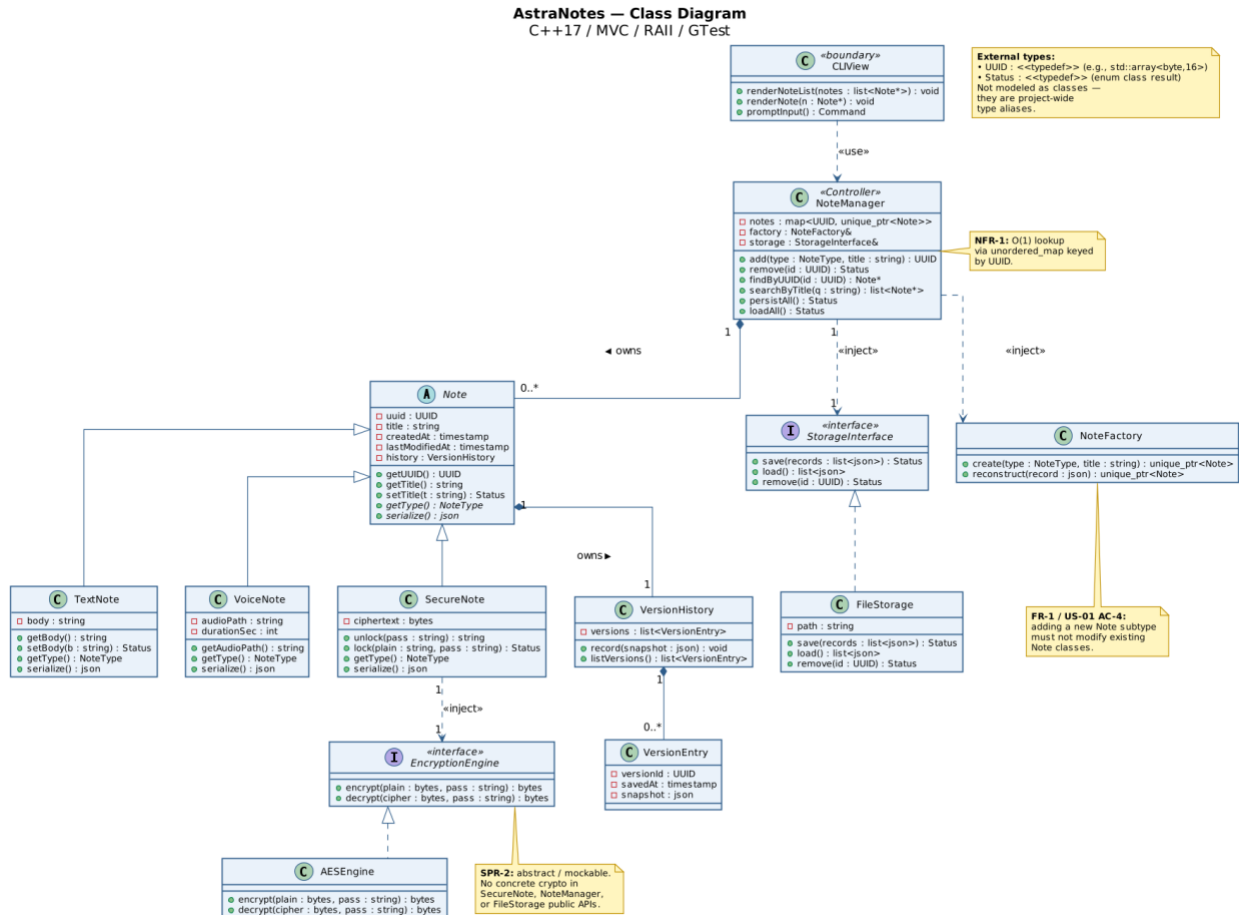


Figure 1. AstraNotes class diagram. C++17 / MVC / RAIL / GTest.

Class Reference

Class	Responsibility	Key Relationships
-------	----------------	-------------------

Note	Abstract base. UUID, title, body, timestamps, VersionHistory.	Extended by TextNote, VoiceNote, SecureNote
TextNote	Stores plain Markdown body.	Extends Note
VoiceNote	Stores audio path and duration; transcription stub.	Extends Note
SecureNote	Holds AES-256 ciphertext only. Plaintext never persisted.	Extends Note; uses EncryptionEngine (injected)
NoteManager	Controller. Owns map<UUID, unique_ptr<Note>>. Orchestrates create/search/persist.	Aggregates NoteFactory&, StorageInterface&; composes map of Notes
NoteFactory	Constructs and reconstructs Note subtypes by NoteType enum. Stateless.	Used by NoteManager
VersionHistory	Maintains ordered list of VersionEntry snapshots for one note.	Owned by each Note instance (1-to-1 composition)
VersionEntry	Immutable record: UUID, timestamp, JSON snapshot.	Owned by VersionHistory
StorageInterface	Pure virtual contract: save, load, remove. Enables mock in tests.	Implemented by FileStorage; injected into NoteManager
FileStorage	Reads/writes notes.json and voice/*.wav via nlohmann/json.	Implements StorageInterface
EncryptionEngine	Interface: encrypt/decrypt. Enables mock per SPR-2.	Implemented by AESEngine; injected into SecureNote
AESEngine	Concrete AES-256-GCM implementation via OpenSSL libcrypto.	Implements EncryptionEngine
CLIView	`«boundary»` View layer. Renders notes and dispatches user actions to the controller. Holds no domain logic.	Depends on NoteManager (Controller); does not access Notes directly

Design Decisions

RAII ownership via map<UUID, unique_ptr<Note>>: NoteManager owns every note through a unique_ptr keyed by UUID. The map gives O(1) lookup for edit, delete, and view operations (NFR-1). When NoteManager is destroyed, all notes are deallocated automatically — no manual cleanup needed.

Injected interfaces for StorageInterface and EncryptionEngine: NoteManager holds StorageInterface& and SecureNote holds EncryptionEngine& — references injected at construction time. This means unit tests can substitute mock implementations without touching NoteManager or SecureNote internals, satisfying SPR-2.

VersionHistory owned by Note, not NoteManager: Each note carries its own history as a 1-to-1 composition. This keeps version data co-located with the note it describes and makes the note lifecycle self-contained — removing a note automatically removes its history.

EncryptionEngine as interface (FR-1 / US-01 AC-4): Adding a new Note subtype cannot require changes to EncryptionEngine, NoteManager, or FileStorage public APIs. The interface boundary enforces this at compile time.

SecureNote stores ciphertext only (SPR-1): SecureNote's ciphertext member is of type bytes, not string. Plaintext is discarded from memory immediately after encryption (enforced in the activity diagram). FileStorage serializes the ciphertext field directly — plaintext never touches disk.

View as a thin boundary (MVC): CLIView is a «boundary» class with no domain logic — it depends on NoteManager and never touches Note instances directly. This keeps the View replaceable (a TUI or GUI front-end could swap in without touching the Model or Controller) and matches the deployment diagram's split between the View component and the rest of the AstraNotes Process.

2. Object Diagram

A snapshot of the running system mid-session: **three notes** in the collection — one TextNote that has been edited once (so its VersionHistory carries two entries), one VoiceNote with a .wav attachment, and one SecureNote whose data members hold ciphertext only. All three Note subtypes from the class diagram are represented, and **each carries its own VersionHistory instance — only `text1`'s is populated this session**. The concrete **AESEngine** instance (not the abstract EncryptionEngine) is shown to confirm the injection pattern works at runtime.

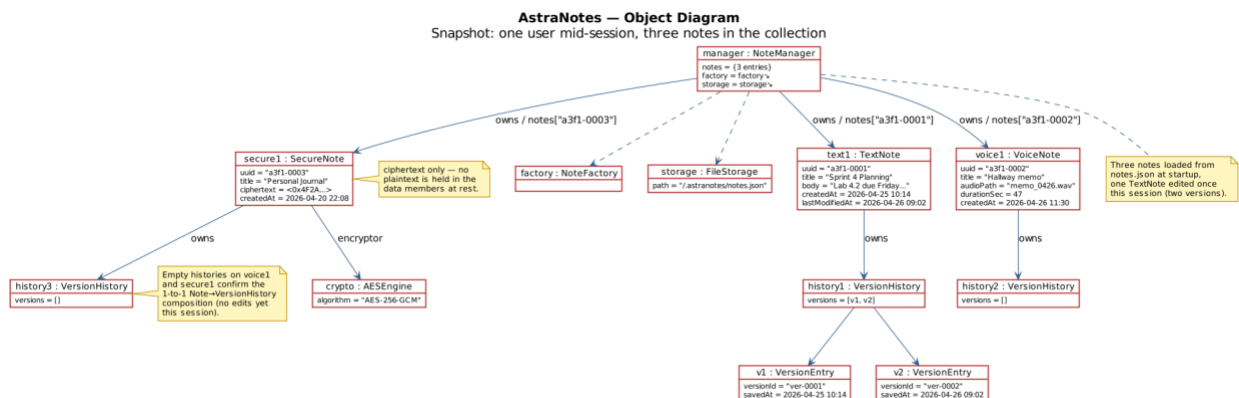


Figure 2. Runtime instance snapshot. One user mid-session, three notes in the collection.

What this snapshot confirms

- **Ownership chain:** manager holds three notes via the UUID map; factory and storage are referenced (not owned) by manager — consistent with the aggregation arrows in Figure 1.
- **VersionHistory location and multiplicity:** every Note instance carries its own VersionHistory — 'history1' hangs off 'text1' with two VersionEntry instances ('v1', 'v2'); 'history2' hangs off 'voice1' and 'history3' hangs off 'secure1', both with empty 'versions = []' lists. Histories never hang off 'manager'. This confirms the 1-to-1 Note→VersionHistory composition declared in Figure 1.

- **SecureNote data discipline:** secure1 shows ciphertext = <0x4F2A...> — bytes, not a readable string. The annotation confirms no plaintext is held in memory at rest (SPR-1).
- **Concrete crypto:** The object is `crypto : AESEngine`, not EncryptionEngine. The interface is injected; the concrete class is what actually runs.

3. Use Case Diagram

User goals only — no class detail. The six primary use cases map directly to the six user stories from Week 2.2 (US-01 through US-06). **Persist Notes to Disk** is modeled as an `«include»` dependency from Create Note, Edit TextNote, and Delete Note — representing FR-5 (durable persistence after every state-changing action). **Encrypt Content** extends Create Note conditionally with the guard [type = SecureNote], and **Unlock SecureNote** extends View Note with [note is SecureNote]. Both extensions map to EncryptionEngine interactions in Figure 1. **Load Notes from Disk** is modeled as an `«include»` dependency from View Note — replacing the informal "enables" relationship from the prior revision with standard UML notation.

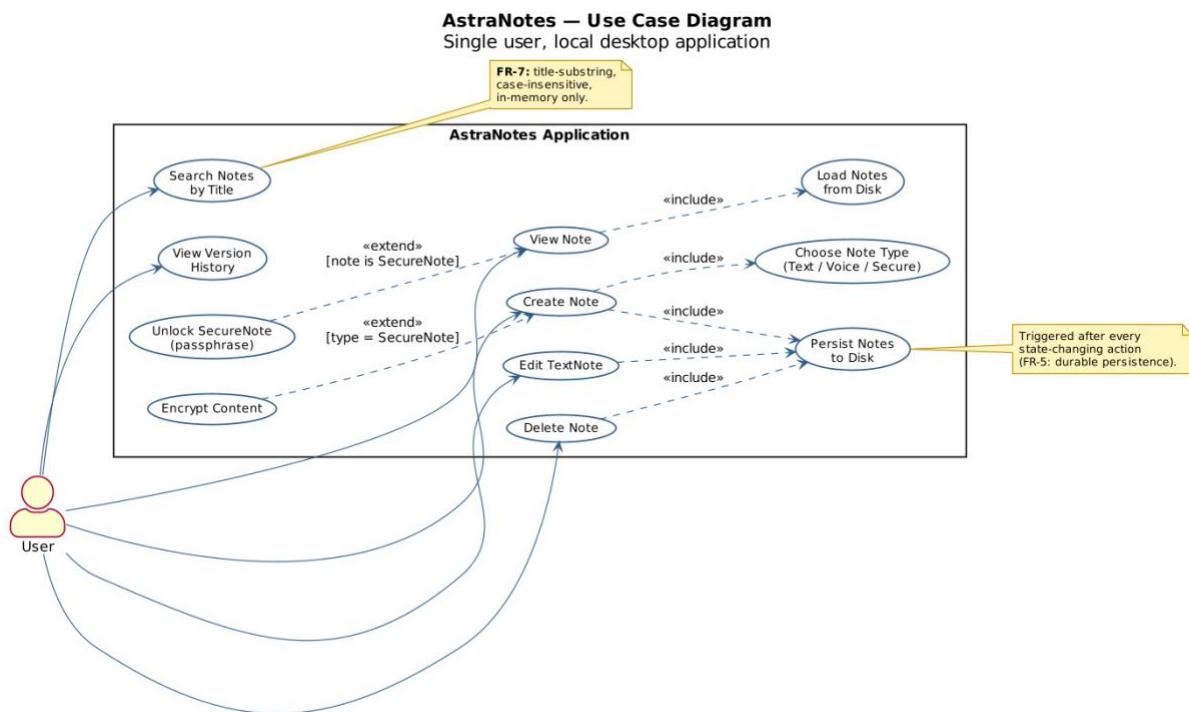


Figure 3. User-facing goals and system actions. «include» = mandatory; «extend» = conditional on note type.

Use Case Summary

- **Load Notes from Disk:** Triggered at startup. FileStorage reads notes.json; NoteFactory reconstructs each Note subtype from its JSON record. Included by View Note as a precondition.
- **View Note:** Includes Load Notes from Disk. If the selected note is a SecureNote, Unlock SecureNote extends this use case — the user provides a passphrase, EncryptionEngine decrypts, and the plaintext is shown only for the session duration.
- **Search Notes by Title:** Title-substring, case-insensitive, in-memory only (FR-7 annotation). NoteManager's searchByTitle method scans the UUID map without touching disk.
- **Create Note:** Always includes Choose Note Type (Text / Voice / Secure) and Persist Notes to Disk (FR-5). If type = SecureNote, also extends to Encrypt Content before persistence.

- **Edit TextNote:** Modifies body and lastModifiedAt, appends a VersionEntry to the note's VersionHistory, then includes Persist Notes to Disk (FR-5).
- **Delete Note:** Removes the note from the UUID map (deallocating via unique_ptr) and includes Persist Notes to Disk to update notes.json atomically.
- **View Version History:** Reads the VersionHistory owned by the selected Note and lists the VersionEntry objects. Read-only — does not trigger persistence.

4. Activity Diagram

One workflow, end to end: **create and persist a SecureNote**. Chosen because it exercises every component in Figure 1 simultaneously and surfaces the two failure modes that matter most — **input validation** (empty title, empty passphrase) and **persistence failure** (atomic write fails, in-memory insert is rolled back). The five swim lanes — User, NoteManager, NoteFactory, SecureNote + EncryptionEngine, and FileStorage — match the component boundaries in the class and deployment diagrams. **Each failure terminator is now distinctly labeled** (`fail: invalid title`, `fail: invalid passphrase`, `fail: encrypt error`, `fail: write error`) so a reviewer can trace any failure mode back to the decision diamond that produced it.

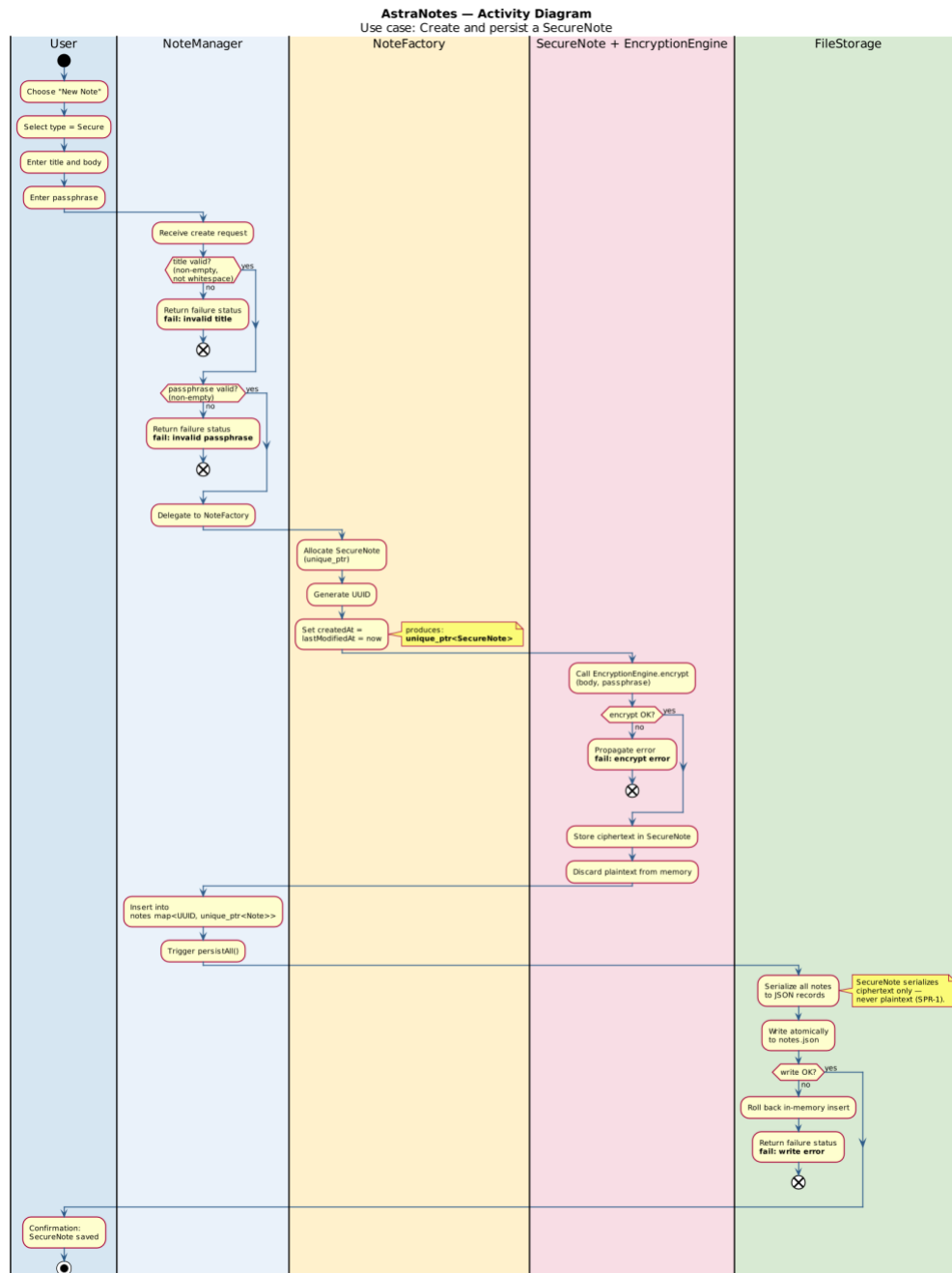


Figure 4. End-to-end SecureNote creation. Validation gates, plaintext discard (SPR-1), and atomic write with rollback.

Workflow Notes

- **Validation gates (NoteManager lane):** Two explicit guards before any allocation — title non-empty and passphrase non-empty. Both return a failure Status immediately, keeping no half-constructed objects on the heap. The two failure terminators are labeled 'fail: invalid title' and 'fail: invalid passphrase'.
- **NoteFactory lane:** Allocates the `unique_ptr<SecureNote>`, generates a UUID, and stamps the creation timestamps. The factory is the only place these values are set. An object-flow note ('produces: `unique_ptr<SecureNote>`') makes the RAII ownership transfer explicit.
- **Plaintext discard (SPR-1):** After `EncryptionEngine.encrypt()` returns, the plaintext is explicitly cleared from memory before the SecureNote `unique_ptr` is returned. The disk record will never contain plaintext

— the annotation links this step directly to SPR-1. An encrypt failure terminates as `fail: encrypt error` before any in-memory insert happens, so no rollback is needed on that branch.

- **Atomic write + rollback (FileStorage lane):** FileStorage serializes all notes to JSON and writes atomically. If the write fails, the in-memory insert into the UUID map is rolled back — the system returns to the state it was in before the create request. The terminator on this branch is labeled `fail: write error`.

5. Deployment Diagram

Local-first, single-user. One compiled C++17 binary on the user's workstation (macOS, Linux, or Windows), two filesystem artifacts (notes.json for all note records, voice/*.wav for VoiceNote attachments), two linked libraries (OpenSSL libcrypto for AES-256-GCM, nlohmann/json header-only for JSON I/O), and a separate `«test infrastructure»` GTest Runner process for CI verification. The C++17 Runtime is now formally an `«executionEnvironment»` rather than a node, nlohmann/json carries the `«header-only»` stereotype, and the compiled binary appears as an explicit `«artifact»` (`astranotes`) inside the AstraNotes Process. No network nodes — that is a charter-driven design decision, not an omission (SPR-1 requires data never leaves the device).

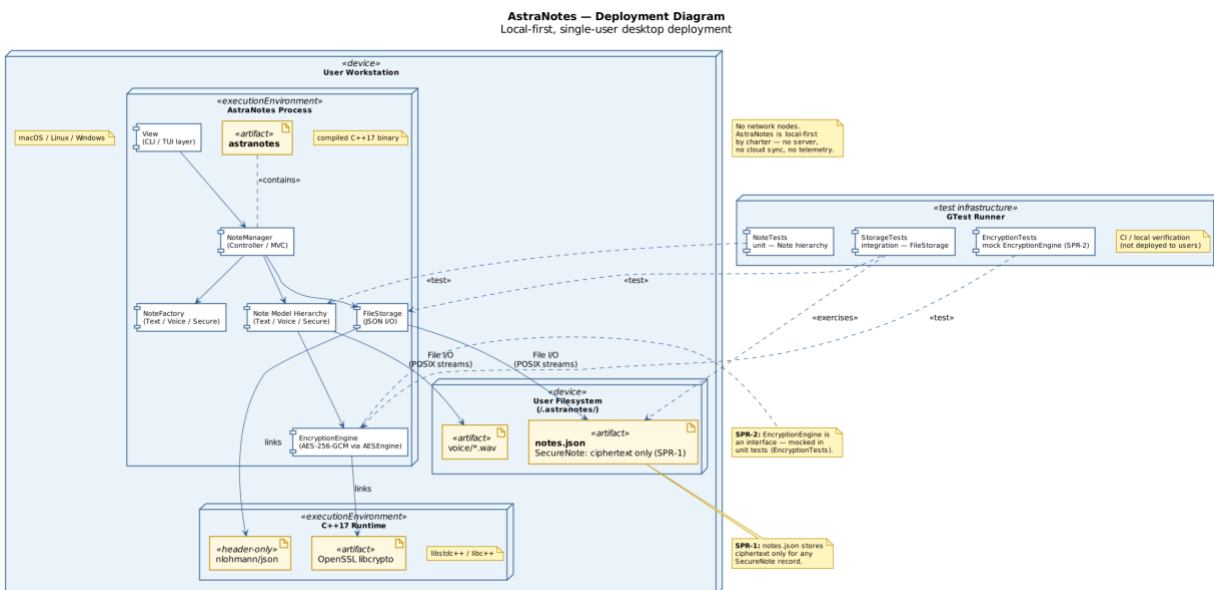


Figure 5. Local-first deployment topology. MVC component split, library dependencies, filesystem artifacts, and test infrastructure.

Node Descriptions

- **User Workstation (device):** The physical or virtual machine. Cross-platform — the C++17 standard and OpenSSL are available on macOS, Linux, and Windows.
- **AstraNotes Process (executionEnvironment):** The compiled binary, shown as the `«artifact»` `astranotes`. Contains the full MVC stack: View (CLI/TUI), NoteManager (Controller), Note Model Hierarchy (Model), FileStorage (JSON I/O), EncryptionEngine / AESEngine, and NoteFactory. The internal arrows show the View → Controller → Model → Factory dependency chain from Figure 1.
- **User Filesystem (~/.astranotes/):** notes.json holds all note records; SecureNote entries contain ciphertext only (SPR-1 annotation). voice/*.wav holds VoiceNote attachments referenced by audioPath in the Note model. Both are reached via labeled `File I/O (POSIX streams)` connectors so the protocol level matches the library link annotations.

- **C++17 Runtime** (`«executionEnvironment»`): OpenSSL libcrypto provides the AES-256-GCM implementation used by AESEngine. nlohmann/json is `«header-only»` and used by FileStorage for serialization / deserialization at compile time.
- **GTest Runner** (`«test infrastructure»`): A separate process running the unit and integration test suite — explicitly stereotyped as test infrastructure (not deployed to users). NoteTests cover the Note hierarchy; StorageTests exercise FileStorage against the real filesystem; EncryptionTests use a mock EncryptionEngine (SPR-2). Connectors to production components carry `«test»` and `«exercises»` stereotypes.

6. Rationale

These five diagrams describe the same AstraNotes project from different angles, and every name, scope assumption, and requirement reference is consistent across all of them. **The class diagram** is the authoritative structural contract: every class, ownership boundary, and interface visible in the other four views resolves back to it — including the full MVC triple (CLIView, NoteManager, Note hierarchy). **The object diagram** validates that contract by showing what three live instances actually look like mid-session, with the UUID map, the per-Note VersionHistory, and the concrete AESEngine injection all directly traceable to Figure 1. **The use case diagram** captures the user's goals and ties each one back to a Week 2.2 user story (US-01 through US-06) and to a requirement ID where applicable (FR-5 for persistence, FR-7 for search). Every use case is supported by at least one structural element in Figure 1. **The activity diagram** closes the loop between structure and behavior by tracing the most cross-cutting workflow end to end — five swim lanes, two validation gates, the SPR-1 plaintext-discard step, and a write failure rollback that would otherwise have to be inferred from the class model. **The deployment diagram** answers where all of this runs: one binary on a user workstation, two filesystem artifacts, two library dependencies, and a GTest Runner — no network, by charter.

The package deliberately omits complexity that AstraNotes does not have. There is no network node, no cloud storage, no second actor, and no SearchIndex class promoted beyond a NoteManager method — adding those would misrepresent the actual project scope. The sticky-note annotations on the class and deployment diagrams tie specific design choices directly to requirement IDs (SPR-1, SPR-2, NFR-1, FR-5, FR-7) so that a future reader can check any constraint against its source.